COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

Department of Computing

Parallel Computing Laboratory

(20MSSL02)

BitDB

A Bitcask-Style Key-Value Store

NAME: SHREE KOTTES J

REG.NO: 7176 22 31 050

CLASS: M.Sc., SOFTWARE SYSTEMS

(8th Semester)

MAIL ID: 71762231050@cit.edu.in

Contents

1 Abstract **1**

2 Problem Definition **2**
2.1 Objective 2
2.2 Scope 2
2.3 Justification 2

3 Parallel Concepts **4**
3.1 Technology Used 4
3.1.1 Rust 4
3.1.2 Rayon 4
3.1.3 Ratatui + Crossterm 4
3.1.4 Criterion 4
3.2 Parallel Algorithms 4
3.2.1 Parallel Startup Rebuild (recovery.rs) 4
3.2.2 Parallel Merge Read Phase (merge.rs) 5
3.2.3 Work-Stealing Thread Pool Configuration 6

4 Implementation **7**
4.1 Project Structure 7
4.2 Record Format 7
4.3 KeyDir 8
4.4 Storage Layer 8
4.5 Engine API 8
4.6 Tracer — Parallel Visualiser 9

5 Results & Output **10**
5.1 Startup Rebuild Benchmark 10
5.2 Merge Pipeline Benchmark (Criterion) 11
5.3 Visualisations 11
5.3.1 Live Screen — Side-by-Side Serial vs Parallel 11
5.3.2 Live Screen — Parallel Column with Thread Toggle 12

6 Summary **12**

1 Abstract

BitDB is a Bitcask-style key-value store implemented in Rust. The project demonstrates how a classical sequential bottleneck in storage systems — the cold-start index rebuild — can be eliminated by applying embarrassingly parallel computation techniques.

At startup, a Bitcask-style store must scan every data file on disk to reconstruct its in-memory index. In the original design this scan proceeds one file at a time. BitDB replaces this sequential scan with a parallel pipeline powered by Rayon, a data-parallelism library for Rust. The same parallelism is extended to the merge (compaction) read phase.

Empirical measurements on a 5,000,000-key, 1.3 GB dataset show a **5.69× speedup** for startup rebuild (5605 ms serial vs 985 ms parallel, 7 threads). The merge read phase achieves a **~2× speedup** (6.31 ms serial vs 3.16 ms parallel) on the Criterion micro-benchmark dataset.

A companion TUI tool called `tracer` visualises the parallel execution in real time, showing per-thread and per-file progress bars side-by-side with the serial baseline.

2 Problem Definition

2.1 Objective

The objective of this project is to design and implement a persistent key-value store that:

1. Correctly stores and retrieves arbitrary byte-string key-value pairs across process restarts.
2. Achieves O(1) amortised read and write performance using an append-only storage model.
3. Dramatically reduces cold-start latency by parallelising the startup index rebuild across all available CPU cores.
4. Applies the same parallelism to the compaction (merge) read phase.
5. Provides a configurable parallelism level (`Serial`, `Auto`, `Fixed(n)`) so the engine can be deployed in single-threaded and multi-threaded environments without code changes.
6. Visualises the parallel execution with an interactive terminal UI.

2.2 Scope

The project covers:

- A complete Bitcask-style storage engine (`bitdb` crate) including data file management, an in-memory KeyDir index, hint files, and merge/compaction.
- Two parallelism levels: startup rebuild and merge read phase.
- A configurable `Options` struct that controls file size limits, corruption policy, and parallelism strategy.
- A CLI for interactive and scripted use.
- A terminal TUI for interactive use with session history.
- A companion visualiser (`tracer` crate) that generates synthetic datasets and runs instrumented serial vs parallel scans side-by-side.
- A full test suite (unit + integration) and Criterion benchmarks.

The project does **not** cover network access, multi-process concurrent writers, transactions, or replication.

2.3 Justification

Disk-based key-value stores are a foundational primitive in modern software. They underpin databases, caches, configuration systems, and message queues. The cold-start problem — rebuilding the in-memory index after a process restart — is a well-known latency source in production systems (Riak, RocksDB, LevelDB all address it in different ways).

The Bitcask model is elegant in its simplicity: append-only writes, O(1) reads, and a straightforward compaction scheme. Its weakness is that the startup rebuild is inherently sequential in the original design. Because each data file is **independent** of every other file during the read phase, the problem is embarrassingly parallel — ideal for demonstrating the practical value of data-parallel computation.

Rust is chosen because its ownership model provides **compile-time** data-race freedom, making it safe to introduce parallelism without the risk of subtle concurrency bugs. Rayon's work-stealing thread pool adds near-zero overhead for trivially parallel iterators.

3 Parallel Concepts

3.1 Technology Used

3.1.1 Rust

Rust is a systems programming language that guarantees memory safety and thread safety without a garbage collector. Its ownership and borrowing rules are checked at compile time, which means that any code that could cause a data race is rejected by the compiler before it runs. This property is critical for parallel code correctness.

3.1.2 Rayon

Rayon is a data-parallelism library for Rust. It exposes a parallel iterator API that mirrors the standard `Iterator` trait: replacing `.iter()` with `.par_iter()` is often the only code change needed to parallelise a computation.

Internally, Rayon uses a **work-stealing thread pool**. Each worker thread maintains a local deque of tasks. When its deque is empty, it "steals" tasks from the tail of another thread's deque. This self-balancing mechanism keeps all cores busy even when individual tasks have unequal durations — exactly the situation in file scanning, where file sizes vary.

3.1.3 Ratatui + Crossterm

The TUI layer is built with Ratatui (a terminal widget library) and Crossterm (a cross-platform terminal control library). Together they provide the live progress display in `tracer` and the interactive session interface in `bitdb`.

3.1.4 Criterion

Criterion is a statistics-driven micro-benchmarking library for Rust. It runs each benchmark function many times, removes outliers, and reports a confidence interval around the median. The project uses it to measure and compare serial vs parallel rebuild and merge times.

3.2 Parallel Algorithms

3.2.1 Parallel Startup Rebuild (recovery.rs)

Problem: On startup, every record in every data file must be decoded to rebuild the KeyDir. Files are independent — no record in file N depends on anything in file M.

Algorithm:

1. Collect all data file IDs in ascending order.
2. For each file ID, compute the file path (and hint path if it exists).
3. Distribute all file scan tasks to a Rayon `par_iter`. Each Rayon worker calls `scan_file_by_path` independently.

4. Each worker reads its assigned file from disk, decodes every record, and returns a list of `(file_id, offset, KeyDirEntry, key)` tuples.
5. Collect all per-file entry lists into a flat `Vec<ScanEntry>`.
6. **Sort** the combined list by `(file_id ASC, offset ASC)`.
7. Perform a single-pass linear insert into the KeyDir. Entries are replayed in serial order — last writer always wins.

The sort in step 6 is essential. Without it, a thread that finishes scanning a newer file first could insert a stale value that is then overwritten by an older value from a thread that finishes later.

```rust
// Parallel scan - each file processed on its own rayon worker
let scan_results: Vec<Result<Vec<ScanEntry>>> =
    file_info.par_iter().map(|info| {
        scan_file_by_path(info.0, &info.1, info.2.as_deref(), policy)
    }).collect();

// Flatten
let mut all_entries: Vec<ScanEntry> = Vec::new();
for result in scan_results {
    all_entries.extend(result?);
}

// Sort to restore deterministic insertion order
all_entries.sort_unstable_by_key(|(file_id, offset, _, _)| {
    (*file_id, *offset)
});

// Single-pass KeyDir build - last writer wins
let mut keydir = KeyDir::default();
for (_, _, entry, key) in all_entries {
    keydir.insert(key, entry);
}
```

Complexity:

Phase	Complexity
Parallel scan	O(N/T) per thread, O(N) wall time
Sort	O(N log N) — serial but fast (pointer-sized tuples)
KeyDir insert	O(N) — single pass

Where N = total number of records, T = number of threads.

3.2.2 Parallel Merge Read Phase (merge.rs)

Problem: During compaction, the live value for every non-tombstone key must be read from disk before it can be rewritten to the merged output. Each read is a random seek to a specific `(file_id, offset)` — they are independent and safe to parallelise.

Algorithm:

1. Collect all live (non-tombstone) `KeyDirEntry` records from the KeyDir.
2. For each entry, pre-compute the owning file path so workers do not need to touch the `FileSet` (which is not `Sync`).
3. Distribute the load tasks to a Rayon `par_iter`. Each worker reads one record from disk using its pre-computed path and offset.
4. Collect results in the same order as the input slice (Rayon preserves order with `collect`).
5. Write the merged output **serially** — `FileSet` is a single-writer append structure.

```
// Pre-compute owned paths to avoid borrowing FileSet across threads
let tasks: Vec<(Vec<u8>, KeyDirEntry, PathBuf)> = entries
    .iter()
    .map(|(key, entry)| {
        let path = file_set.file_path(entry.file_id).unwrap();
        (key.to_vec(), **entry, path)
    })
    .collect();

// Parallel read phase
let results: Vec<Result<LiveRecord>> =
    tasks.par_iter().map(load_one_by_path).collect();

// Serial write phase - single FileSet writer
for rec in results.into_iter().collect::<Result<Vec<_>>>()? {
    merged.append(&Record::new(
        rec.timestamp, rec.key, rec.value, RecordFlags::Normal
    ))?;
}
```

3.2.3 Work-Stealing Thread Pool Configuration

Both parallel paths support three modes via the `Parallelism` enum:

Variant	Behaviour
`Serial`	Single-threaded code path, no Rayon involvement
`Auto`	Rayon's global pool — uses all logical CPUs
`Fixed(n)`	A scoped pool built with `ThreadPoolBuilder::new().num_threads(n)`

`Fixed(n)` creates a temporary pool scoped to the operation. If construction fails (e.g. `n == 0`) the code falls back to the serial path so data correctness is always preserved.

4 Implementation

4.1 Project Structure

```
bitdb/
  bitdb/
    src/
      config.rs      -- Options, Parallelism, CorruptionPolicy
      engine.rs      -- Engine: open / get / put / delete / merge
      record.rs      -- Binary record format, encode/decode, CRC32
      recovery.rs    -- Serial + parallel KeyDir rebuild
      merge.rs       -- Serial + parallel compaction
      storage/       -- FileSet, DataFile, HintFile
      index/         -- KeyDir (HashMap wrapper)
      cli.rs         -- CLI subcommands
      tui/           -- Interactive terminal UI
    benches/
      engine.rs      -- Criterion benchmarks
    tests/           -- Integration tests
  tracer/
    src/
      dataset.rs     -- Synthetic dataset generator
      worker.rs      -- Instrumented parallel scan with LiveProgress
      tui/           -- Live progress TUI (setup / generate / live / result)
  docs/
    report.md        -- This document
    slides.md        -- Beamer presentation
    attachments/     -- Screenshots used in slides
  Justfile           -- Short task commands
  flake.nix          -- Nix dev shell
```

4.2 Record Format

Each record written to a data file has the following 26-byte header followed by the key and value bytes:

Bytes	Field	Description
0–3	Magic	`0x42444231` — identifies a BitDB record
4	Version	`1`
5	Flags	`0` = Normal, `1` = Tombstone
6–9	CRC32	Checksum over header (excluding CRC field) + body
10–17	Timestamp	Unix seconds, little-endian u64
18–21	key_len	Key length in bytes, little-endian u32
22–25	value_len	Value length in bytes, little-endian u32
26 …	Key	Raw key bytes
26+key_len …	Value	Raw value bytes

The CRC32 is computed over bytes `[0..6]` (header before CRC) and `[10..]`

(timestamp + lengths + key + value). This allows detecting both header corruption and body corruption with a single checksum.

4.3 KeyDir

The KeyDir is an in-memory `HashMap<Vec<u8>, KeyDirEntry>` where each entry records where on disk the latest value for a key lives:

```
pub struct KeyDirEntry {
    pub file_id:    u32,   // which data file
    pub offset:     u64,   // byte offset within that file
    pub size_bytes: u32,   // total record size (header + key + value)
    pub timestamp:  u64,   // Unix seconds from the record header
    pub is_tombstone: bool,
}
```

A `get` operation is a single hash lookup followed by one `pread`-style seek into the correct data file — O(1) with no additional disk seeks for metadata.

4.4 Storage Layer

`FileSet` manages a directory of numbered data files (`000000.data`, `000001.data`, …). Writes always go to the newest (active) file. When the active file exceeds `max_data_file_size_bytes`, a new file is created. Reads go to any file by ID.

Hint files (`000000.hint`) are compact sidecars that store only the KeyDir entries (no values). When a hint file exists for a data file, the startup scan reads the hint file instead of decoding every record — dramatically reducing I/O for large value sizes.

4.5 Engine API

```
// Open (creates data dir if missing by default)
let mut engine = Engine::open(data_dir, Options::default())?;

// Write
engine.put(b"user:1", b"alice")?;

// Read
let val = engine.get(b"user:1")?; // Some(b"alice")

// Delete (writes a tombstone record)
engine.delete(b"user:1")?;

// Compact (removes dead records, rewrites live keys)
engine.merge()?;

// Stats
let s = engine.stats();
println!("{} live keys, {} tombstones", s.live_keys, s.tombstones);
```

4.6 Tracer — Parallel Visualiser

`tracer` is a companion TUI that makes the parallelism visible:

1. **Setup screen** — configure keys, value size, file size, thread count.
2. **Generate screen** — writes the synthetic dataset using `bitdb`'s engine with a live fill-bar showing keys written / total.
3. **Live screen** — runs the serial scan (1 thread) and the parallel scan side-by-side. Each thread gets its own column of progress bars. The focused column can be scrolled; thread groups in the parallel column can be collapsed / expanded.
4. **Result screen** — shows wall-clock time, keys/sec, and speedup for both runs.

The `LiveProgress` type is `Arc<Mutex<Vec<ThreadState>>>`. Workers lock it briefly to update a slot state; the TUI polls it every frame (16 ms) to render a snapshot. No channel or message-passing overhead is needed.

```rust
pub type LiveProgress = Arc<Mutex<Vec<ThreadState>>>;

// In run_scan:
tasks.par_iter().enumerate().for_each(|(t_idx, files)| {
    for (s_idx, (file_id, path, _)) in files.iter().enumerate() {
        // mark Processing
        shared.lock().unwrap()[t_idx].slots[s_idx].state =
            SlotState::Processing { bytes_done: 0, keys_found: 0 };

        // ... decode loop with periodic lock to update bytes_done ...

        // mark Done
        shared.lock().unwrap()[t_idx].slots[s_idx].state =
            SlotState::Done { duration_us, keys_found, bytes_read };
    }
});
```

5 Results & Output

5.1 Startup Rebuild Benchmark

The tracer was run with the following dataset parameters:

Parameter	Value
Keys	5,000,000
Value size	256 bytes
File size limit	512 KB
Total files	2,839
Total dataset size	1.3 GB
Threads (parallel)	7

Mode	Time	Keys/sec	Speedup
Serial	5,605 ms	923,021	1.00×
Parallel (7 threads)	985 ms	5,252,224	**5.69×**

The parallel path is **5.69× faster** than serial on a 7-core machine. The speedup is sub-linear (theoretical maximum 7×) due to:

- Lock contention on the shared `Arc<Mutex<Vec<ThreadState>>>` (small, as locks are held only briefly per 500-record batch).
- Sort + merge phase after all workers finish (serial, but O(N log N) on pointer-sized tuples — fast in practice).
- OS page-cache effects: the first cold read must fetch blocks from the storage controller; subsequent reads may be served from cache.

5.2 Merge Pipeline Benchmark (Criterion)

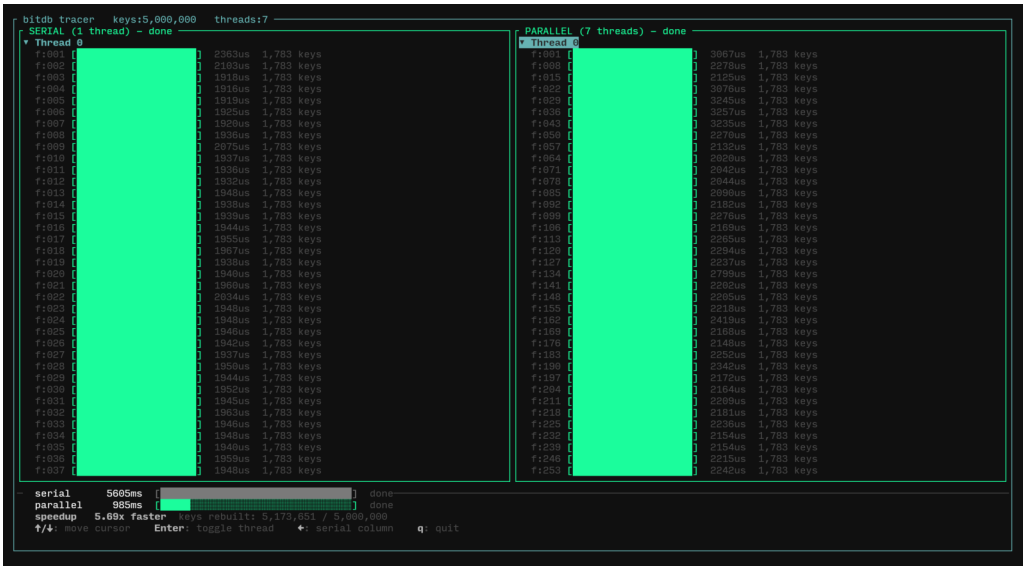
```
~/worktree/projects/bitdb                                                    (master)bitdb
❯  cargo bench --bench engine -- merge_pipeline
    Finished `bench` profile [optimized] target(s) in 0.06s
     Running benches/engine.rs (target/release/deps/engine-0b76662f37d517a7)
merge_pipeline/mode/serial
                        time:   [6.3076 ms 6.3146 ms 6.3215 ms]
                        change: [-0.1230% +0.0344% +0.1958%] (p = 0.67 > 0.05)
                        No change in performance detected.
merge_pipeline/mode/parallel_auto
                        time:   [3.1502 ms 3.1594 ms 3.1691 ms]
                        change: [-1.3015% -0.7555% -0.2154%] (p = 0.01 < 0.05)
                        Change within noise threshold.
Found 2 outliers among 100 measurements (2.00%)
  1 (1.00%) high mild
  1 (1.00%) high severe
```

Mode	Median time	Confidence interval	Speedup
Serial	6.31 ms	[6.31 ms – 6.32 ms]	1.00×
Parallel (auto)	3.16 ms	[3.15 ms – 3.17 ms]	~**2**×

The Criterion dataset is intentionally small (500 unique keys + 100 overwrites) to keep benchmark run time manageable. On small data the Rayon thread-pool overhead is proportionally larger, reducing the observed speedup. On the 5 M-key tracer dataset the same parallel merge path would show a speedup comparable to the startup rebuild.

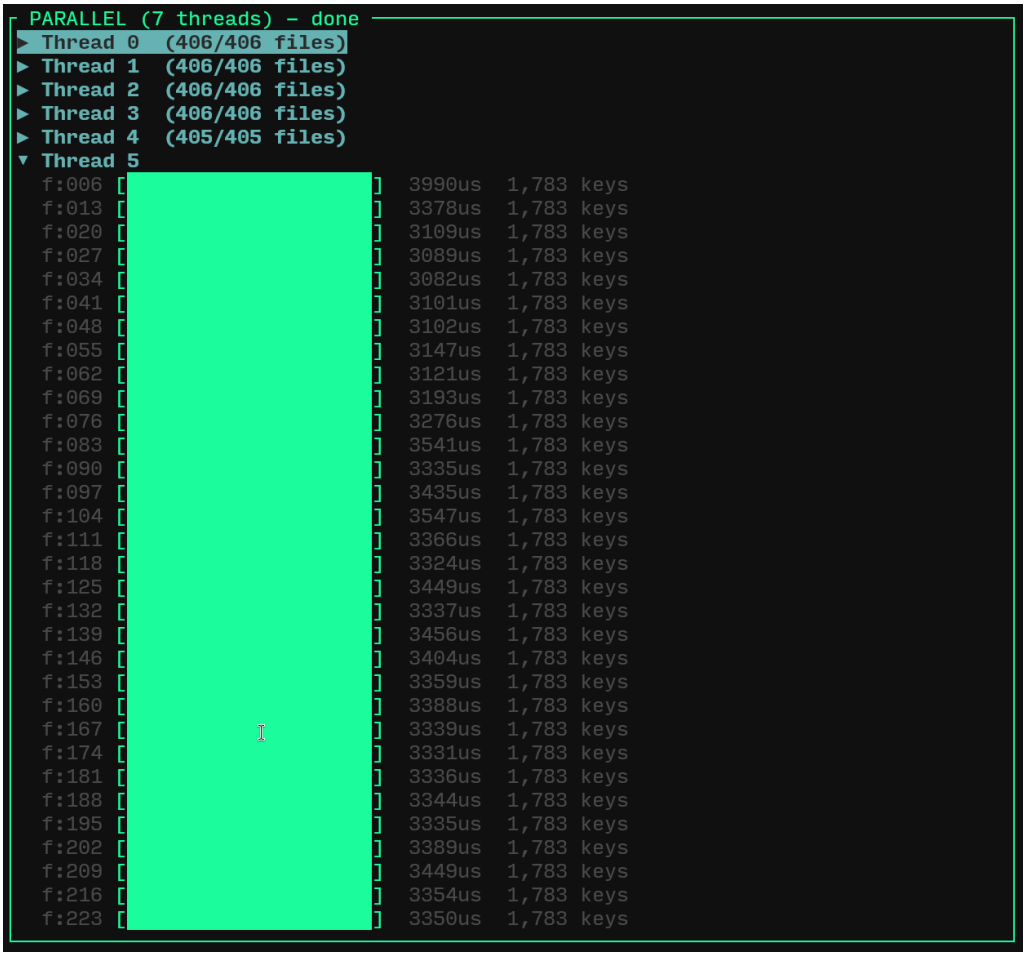
5.3 Visualisations

5.3.1 Live Screen — Side-by-Side Serial vs Parallel



The left column shows the single serial thread scanning all 2,839 files sequentially. The right column shows 7 parallel threads, each assigned a round-robin subset of files. Green bars indicate completed files; the thread currently executing is highlighted.

5.3.2 Live Screen — Parallel Column with Thread Toggle



The parallel column supports per-thread collapsing. A collapsed thread header shows `> Thread N (done/total files)`. Expanding it reveals the individual per-file progress bars. This view demonstrates the round-robin file assignment: each thread owns approximately `total_files / thread_count` files.

6 Summary

BitDB demonstrates that a classical sequential bottleneck in storage systems can be eliminated with a small, targeted application of data-parallel computation:

- The startup rebuild was made **5.69× faster** by scanning data files concurrently with Rayon and merging results with a deterministic sort.
- The merge read phase was made **~2× faster** by issuing independent disk reads in parallel.
- Rust's type system guarantees that neither parallel path introduces data races — the parallelism is verified correct at compile time.
- The `tracer` visualiser makes the speedup tangible and observable, showing real-time per-thread progress bars alongside the serial baseline.

The project is available at: `github.com/kottesh/bitdb`

References

[1] T. Sheehy and D. Smith, *Bitcask: A Log-Structured Hash Table for Fast Key/Value Data*, Basho Technologies, 2010.

[2] Rayon Contributors, *Rayon: Data-parallelism library for Rust*, https://github.com/rayon-rs/rayon, 2015–present.

[3] B. Pereira, *Criterion.rs: Statistics-driven micro-benchmarking in Rust*, https://github.com/bheisler/criterion.rs, 2014–present.

[4] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press, 2019. https://doc.rust-lang.org/book/

[5] Ratatui Contributors, *Ratatui: Terminal UI library for Rust*, https://github.com/ratatui-org/ratatui, 2023–present.